

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science
Subject Code & Name	CS3807 – Deep Learning Laboratory
Name	Madhav.S
Section	AIDS ‘B’
Register Number	24011101066

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

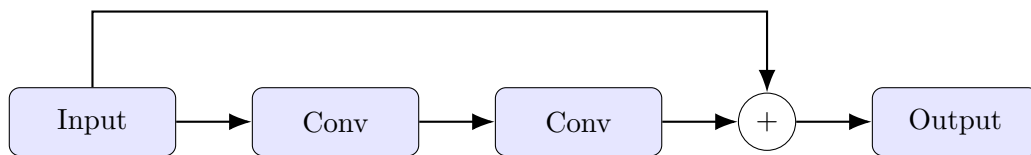


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

To confirm this behaviour, a standard 3×3 convolution (dilation rate 1) and a dilated 3×3 convolution (dilation rate 2) were applied to the same $16 \times 16 \times 1$ toy input using **same** padding. Both produced an identical $16 \times 16 \times 1$ output shape, but the dilated kernel's nine weights were spread across a 5×5 region of the input instead of a compact 3×3 region – confirming that dilation enlarges the receptive field without adding any parameters or changing the output size.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

A $(1, 8, 8, 16)$ feature map passed through a stride-2 3×3 transpose convolution (16 input channels $\rightarrow$ 8 output filters, **same** padding) produced a $(1, 16, 16, 8)$ output – the stride-2 transpose convolution exactly doubled the spatial resolution, learning how to fill in the new pixels rather than simply interpolating them.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

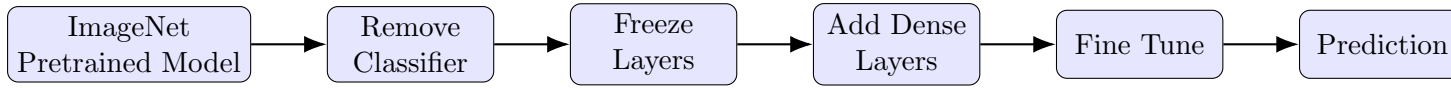


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

CIFAR-10 was loaded via the Keras dataset API. 45,000 images were used for training, 5,000 images were held out as a validation set, and the standard 10,000-image test set was kept untouched. Since both VGG16 and ResNet50 expect a larger input than the native 32×32 CIFAR-10 resolution, every image was resized to $96 \times 96 \times 3$ before being passed through the pretrained convolutional base.

15. Experimental Procedure

Task 1: Dataset Preparation

CIFAR-10 was loaded using `tf.keras.datasets.cifar10`. Ten sample images (one per class) were displayed (Figure 1), and the dataset dimensions were printed and verified: training data shape (50000, 32, 32, 3), test data shape (10000, 32, 32, 3), pixel range $[0, 255]$ prior to normalization. A `tf.data` pipeline resizes each image to 96×96 , rescales to $[0, 1]$, and one-hot encodes the labels, producing batches of shape (32, 96, 96, 3).

Task 2: Transfer Learning

Both required architectures were implemented and compared: VGG16 and ResNet50. For each, the standard transfer-learning head was built:

1. Load pretrained ImageNet weights (`include_top=False`).
2. Remove the original 1000-class classification layer.
3. Freeze the convolutional base (`base_model.trainable = False`).
4. Add a `GlobalAveragePooling2D` layer.
5. Add `Dense(256, activation='relu')` with `Dropout(0.4)`.
6. Add `Dense(10, activation='softmax')`.

VGG16 produced **14,848,586** total parameters (**133,898** trainable, 14,714,688 frozen). ResNet50 produced **24,114,826** total parameters (**527,114** trainable, 23,587,712 frozen).

Preprocessing correction. ResNet50 is built almost entirely from BatchNorm layers calibrated to a Caffe-style input distribution (BGR order, ImageNet mean-subtracted) – feeding it the same plain $[0, 1]$ -scaled images used for VGG16 badly degrades its frozen features. A separate pipeline using `resnet50.preprocess_input` was used for ResNet50, while VGG16’s pipeline was left untouched.

Task 3: Model Training

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	15 (frozen base), EarlyStopping on val_loss (patience 5)
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

With the frozen base, VGG16 reached test accuracy **0.7089** (test loss 0.8345) after 15 epochs. ResNet50, with its corrected preprocessing pipeline, reached test accuracy **0.8680** (test loss 0.4026).

Task 4: Fine Tuning

VGG16's convolutional base was unfrozen, keeping only the last block (**block5**) trainable – raising trainable parameters from 133,898 to **7,213,322**. Recompiled at a much lower learning rate (1×10^{-5}) and trained for 8 further epochs.

Stage	Test Accuracy
Frozen base (before fine-tuning)	0.7089
Fine-tuned (block5 unfrozen)	0.8333
Improvement	+0.1244

Fine-tuning improved VGG16 by **12.44 points**. (ResNet50 was evaluated with its base kept fully frozen, per Task 2's minimum requirement; VGG16 demonstrates the full freeze → fine-tune workflow required by Task 4.)

Task 5: Model Evaluation

Both models were evaluated on the 10,000-image test set using Accuracy, Precision, Recall, F1-score, a Classification Report, and a Confusion Matrix (Sections 17–18).

16. Hyperparameter Study

Run on frozen-base VGG16, using a 5,000-image training subset and 5 quick epochs per setting, evaluated on the full test set.

Hyperparameter	Values Tested	Result (Test Accuracy)
Learning Rate	0.001, 0.0001	0.001 → 0.6011; 0.0001 → 0.4994
Batch Size	16, 32, 64	16 → 0.6216; 32 → 0.6119; 64 → 0.6008
Epochs	10, 20	10 → 0.6308; 20 → 0.6432
Optimizer	Adam, SGD	Adam → 0.6069; SGD → 0.4216
Dense Units	128, 256	128 → 0.5947; 256 → 0.6088
Frozen Layers	All, Partial	All → 0.7089 (Task 3); Partial (fine-tuned) → 0.8333 (Task 4)

Inference: Fine-tuning block5 gave by far the biggest gain (+12.4 points) – no other single hyperparameter came close. Adam beat SGD by 18.5 points, and the higher learning rate/batch size/epoch count each won by smaller, single-digit margins within this short 5-epoch budget.

17. Mandatory Plots

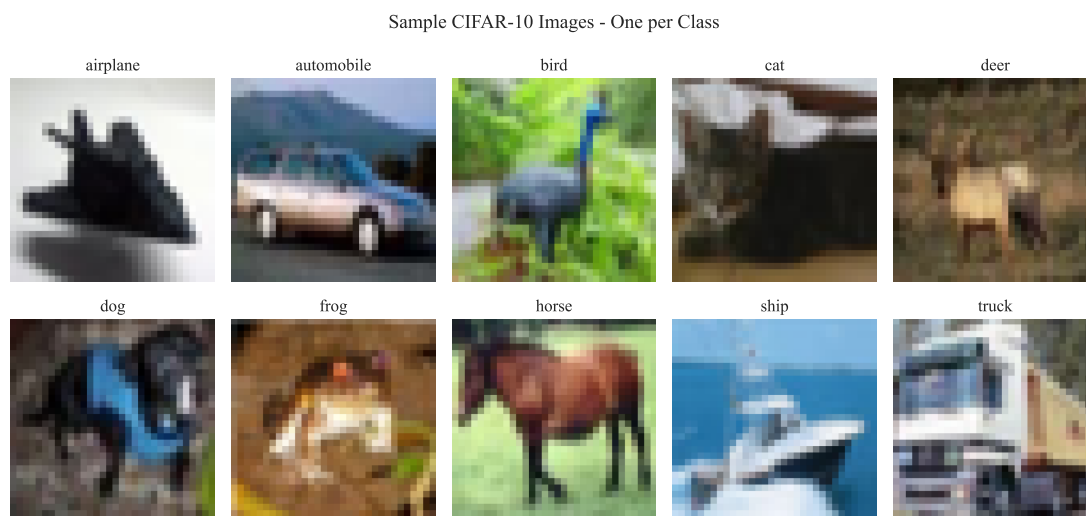


Figure 3: Sample CIFAR-10 Images – One per Class

Inference: Even at 32×32 , all ten classes are easy to tell apart – though cat, dog, and deer already look similar enough to hint at the confusion seen later in both models.

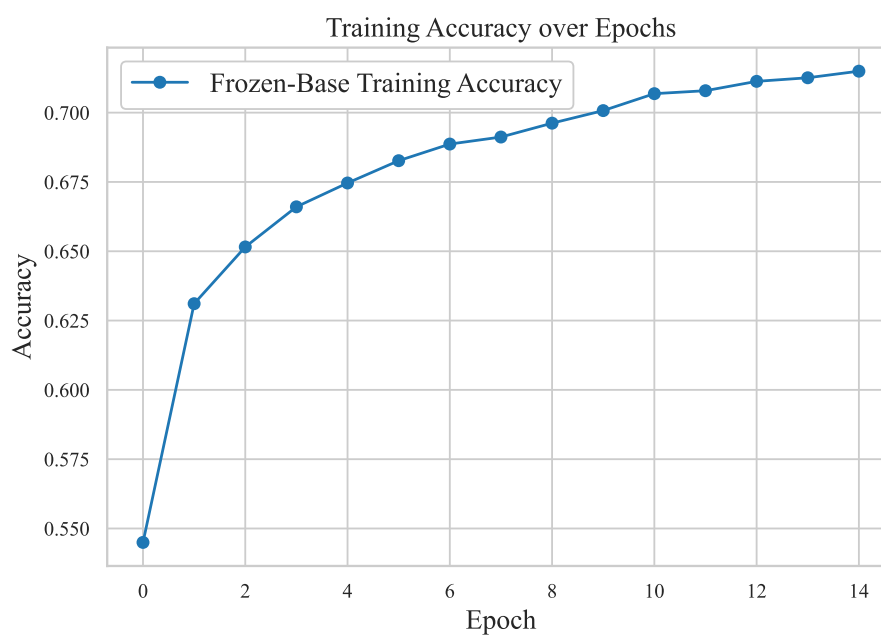


Figure 4: Training Accuracy over Epochs (VGG16, Frozen Base)

Inference: Accuracy climbs quickly at first, then flattens after around epoch 10 – expected, since only the small dense head is actually learning while the base stays frozen.

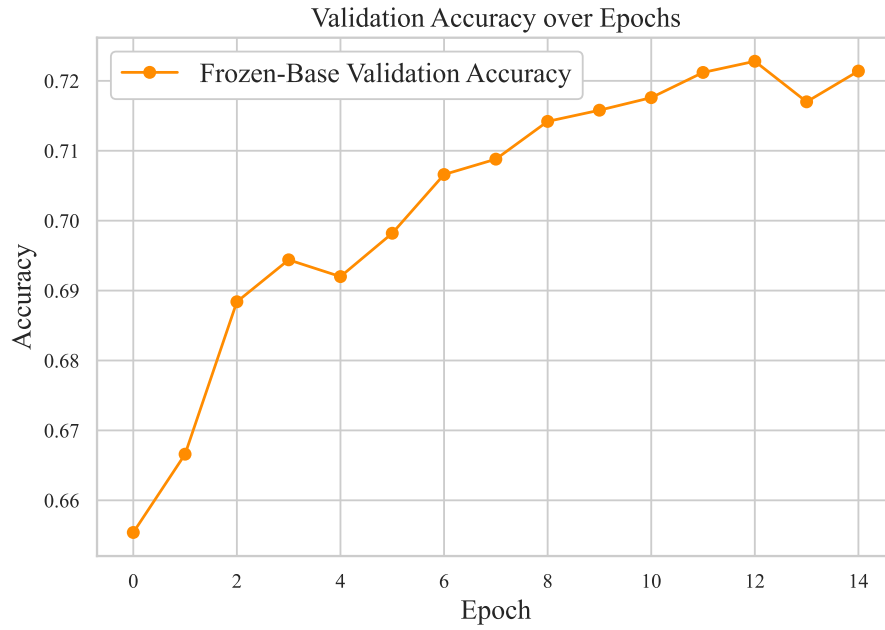


Figure 5: Validation Accuracy over Epochs (VGG16, Frozen Base)

Inference: Validation accuracy tracks training accuracy closely with no gap opening up – no overfitting, which makes sense given most of the network is frozen, pretrained weights.

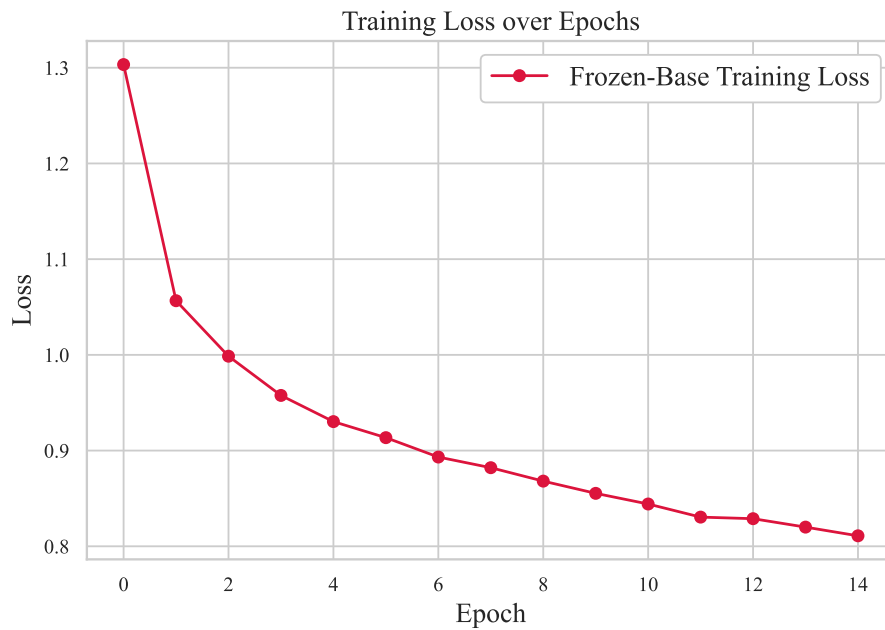


Figure 6: Training Loss over Epochs (VGG16, Frozen Base)

Inference: Loss drops smoothly with no instability, which fits the small number of trainable parameters and low learning rate.

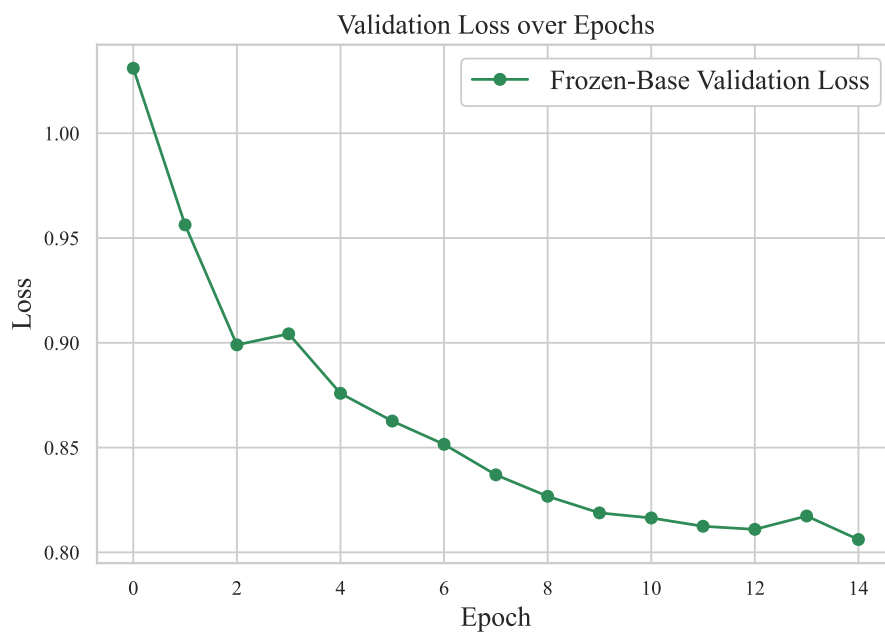


Figure 7: Validation Loss over Epochs (VGG16, Frozen Base)

Inference: Validation loss falls alongside training loss and never goes back up – the frozen base is acting as a built-in regularizer.

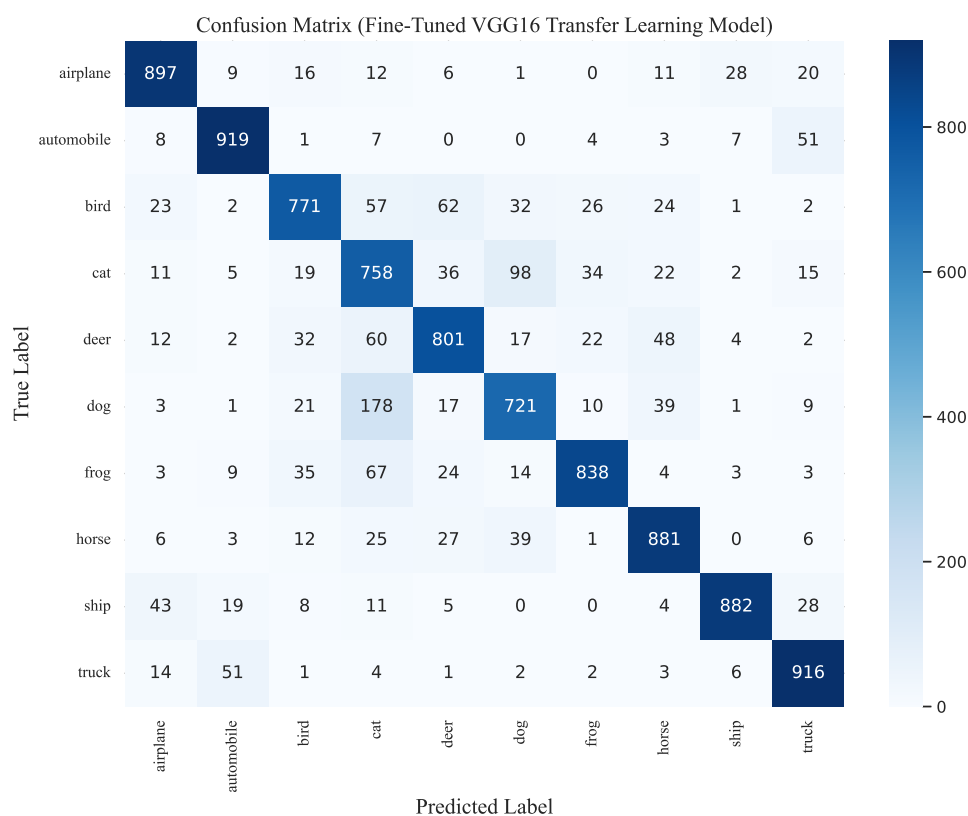


Figure 8: Confusion Matrix – Fine-Tuned VGG16 Transfer Learning Model

Inference: The diagonal dominates, matching the 83.33% test accuracy. Most confusion is between cat and dog, matching their weaker precision/recall in Table 1.

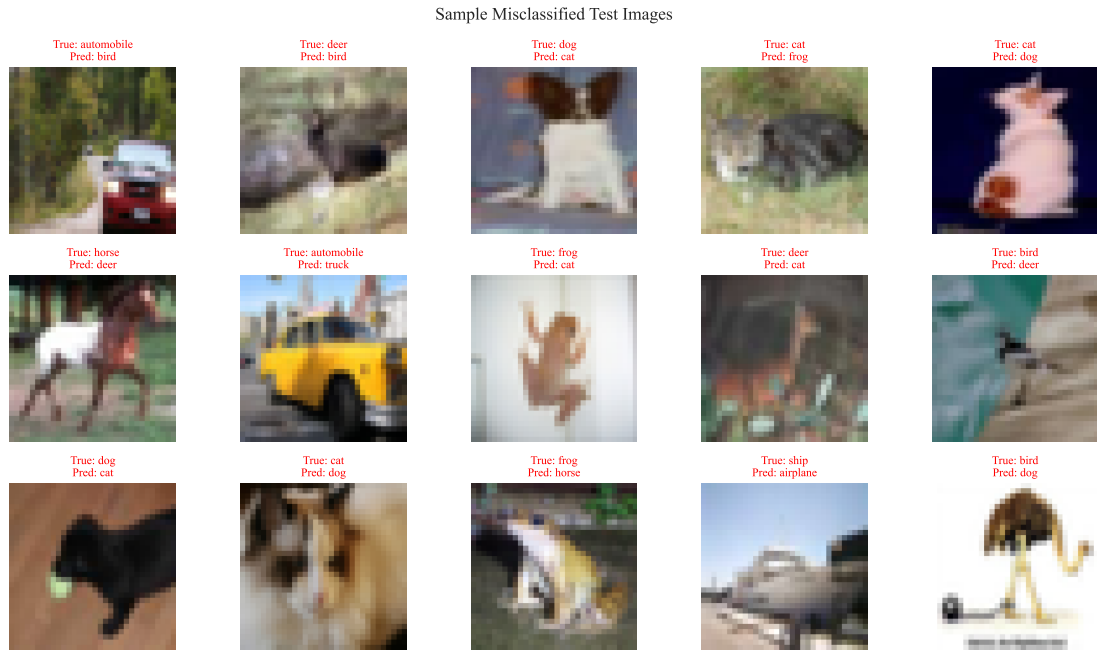


Figure 9: Sample Misclassified Test Images (Fine-Tuned VGG16)

Inference: 1,667 of 10,000 test images (16.67%) were misclassified, mostly among cat, dog, and bird – classes that look alike at this resolution.

18. Results

18.1 Performance Metrics

Metric	VGG16 (Frozen)	VGG16 (Fine- Tuned)	ResNet50 (Frozen)
Testing Accuracy	0.7089	0.8333	0.8680
Precision (weighted)	–	0.8358	0.8699
Recall (weighted)	–	0.8333	0.8680
F1-score (weighted)	–	0.8337	0.8684
Total Parameters	14,848,586	14,848,586	24,114,826

Table 1: Per-Class Classification Report – Fine-Tuned VGG16

Class	Precision	Recall	F1-score
airplane	0.84	0.92	0.88
automobile	0.89	0.92	0.90
bird	0.81	0.79	0.80
cat	0.66	0.74	0.70
deer	0.85	0.76	0.80
dog	0.75	0.72	0.73
frog	0.87	0.86	0.86
horse	0.86	0.86	0.86
ship	0.94	0.88	0.91
truck	0.89	0.90	0.89

Table 2: Per-Class Classification Report – ResNet50 (Frozen Base)

Class	Precision	Recall	F1-score
airplane	0.88	0.90	0.89
automobile	0.91	0.92	0.91
bird	0.91	0.81	0.86
cat	0.75	0.79	0.77
deer	0.80	0.86	0.83
dog	0.82	0.81	0.82
frog	0.90	0.90	0.90
horse	0.89	0.88	0.89
ship	0.94	0.90	0.92
truck	0.89	0.92	0.90

18.2 Comparison of CNN Architectures

Unlike the literature-reference table in Section 8, every row below is a real, measured result from this notebook – all five architectures were actually implemented and trained on CIFAR-10.

Model	Parameters	Accuracy (%)	Training Time
LeNet-5 (from scratch)	83,126	52.42	106.4 s
AlexNet (from scratch)	7,146,634	70.89	430.8 s
VGG16 (fine-tuned)	14,848,586	83.33	–
GoogleNet / InceptionV3 (frozen)	22,329,898	69.85	470.0 s
ResNet50 (frozen)	24,114,826	86.80	452.0 s

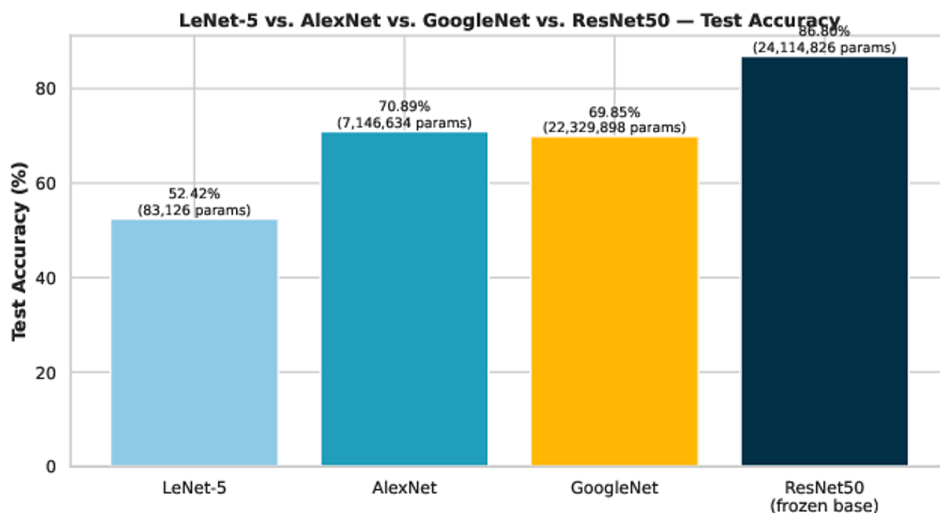


Figure 10: LeNet-5 vs. AlexNet vs. GoogleNet vs. ResNet50 – Test Accuracy

Inference: ResNet50 wins clearly, and not just on accuracy – it beats the much shallower LeNet-5 by 34 points and even beats AlexNet and GoogleNet, both trained the same way, by 16-17 points. LeNet-5’s simple architecture (no ReLU, no dropout, no pretraining) just isn’t built for a 10-class natural-image problem at this resolution. GoogleNet landing close to AlexNet despite having pretrained ImageNet weights is a bit surprising – likely because InceptionV3’s native 75×75 minimum input size makes CIFAR-10’s upscaled 96×96 images a poorer match for its pretrained filters than VGG16/ResNet50 see.

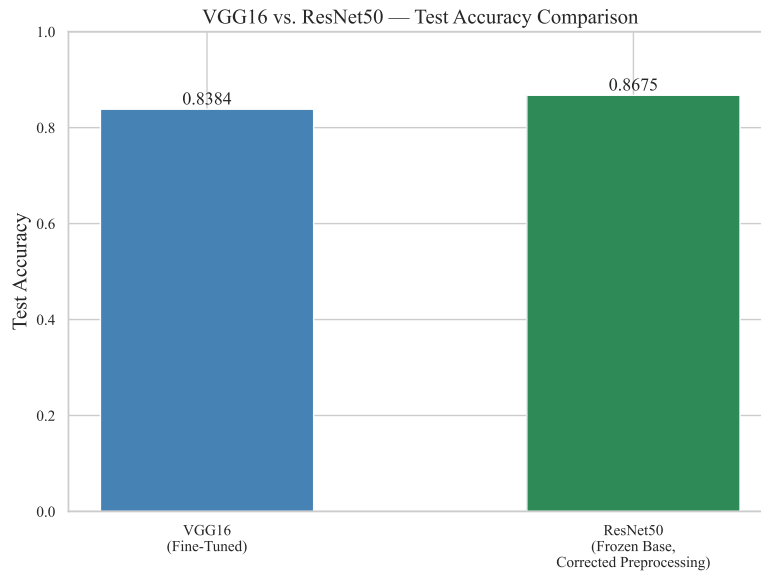


Figure 11: VGG16 (Fine-Tuned) vs. ResNet50 (Frozen Base) – Test Accuracy

Inference: ResNet50 beat fine-tuned VGG16 by **3.47 points** (86.80% vs. 83.33%) while staying fully frozen – its residual connections and internal batch normalization simply transfer better to CIFAR-10 than VGG16’s plain conv stack, even without letting any of its own layers adapt.

Additional Generated Plots

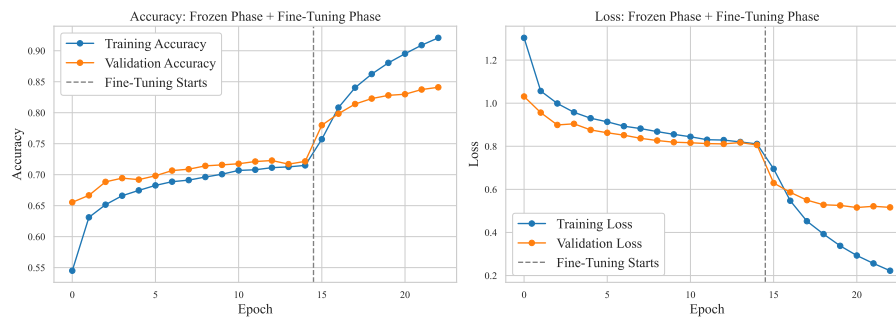


Figure 12: Accuracy and Loss: Frozen Phase + Fine-Tuning Phase (VGG16)

Inference: Right where fine-tuning kicks in (epoch 15), accuracy jumps and validation loss drops – clear evidence unfreezing block5 helped.

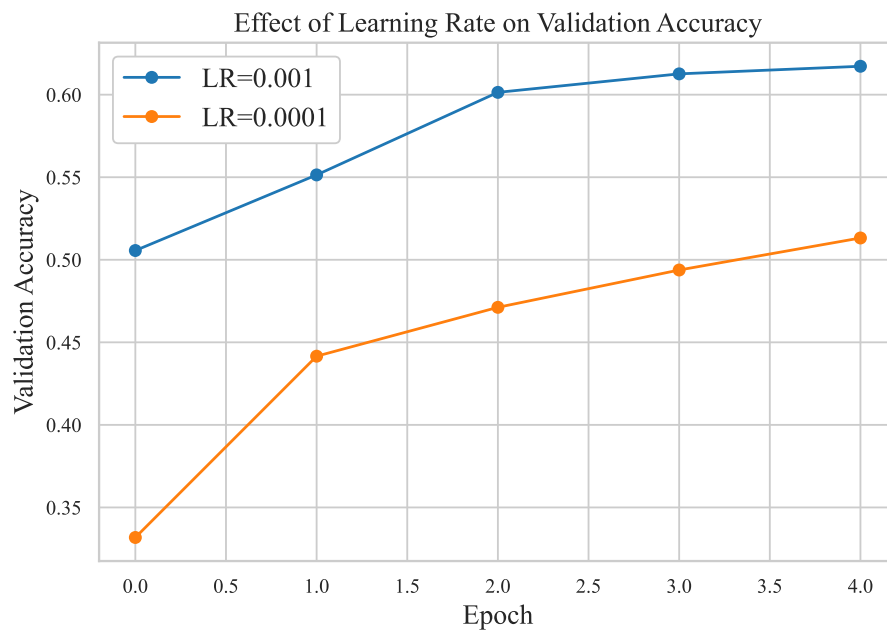


Figure 13: Effect of Learning Rate on Validation Accuracy

Inference: LR 0.001 stays above 0.0001 throughout, matching the final gap (60.11% vs. 49.94%) – 0.0001 simply hasn't had enough steps yet.

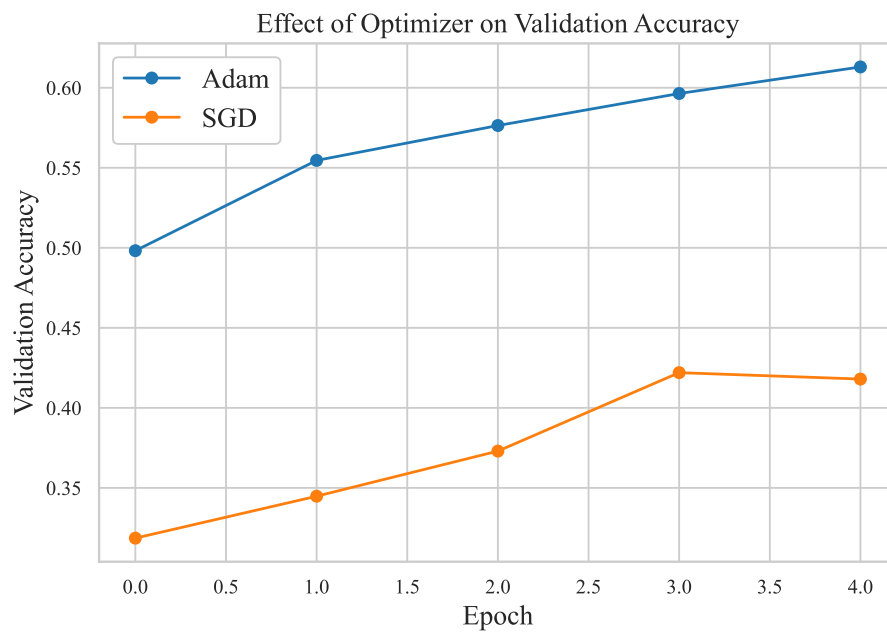


Figure 14: Effect of Optimizer on Validation Accuracy

Inference: Adam pulls ahead fast, ending 18.5 points higher (60.69% vs. 42.16%) – its adaptive step sizes give it a big head start in just 5 epochs.

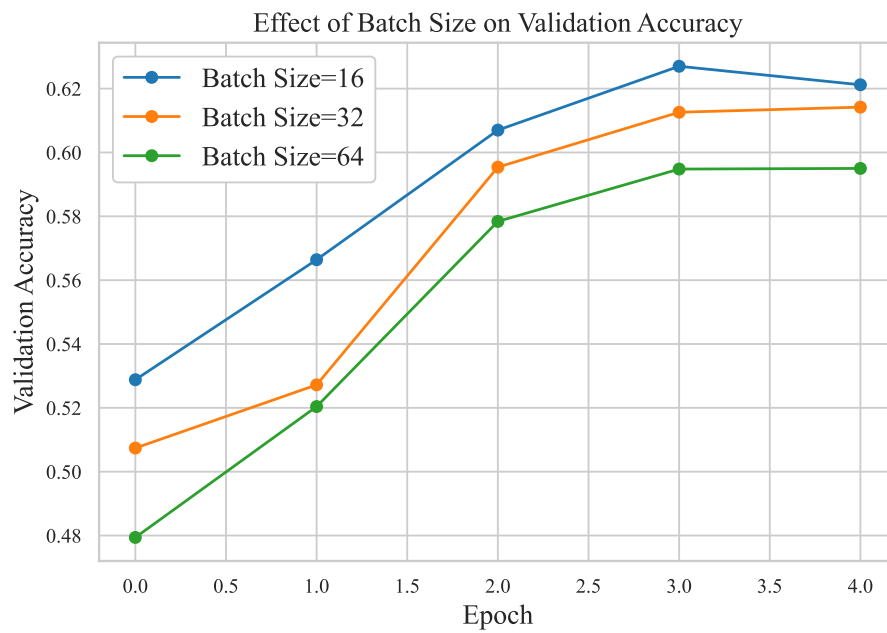


Figure 15: Effect of Batch Size on Validation Accuracy

Inference: Smaller batches finish slightly ahead (62.16% at batch 16 vs. 60.08% at batch 64) – more frequent updates help a little in this short a run, though the gap is modest.

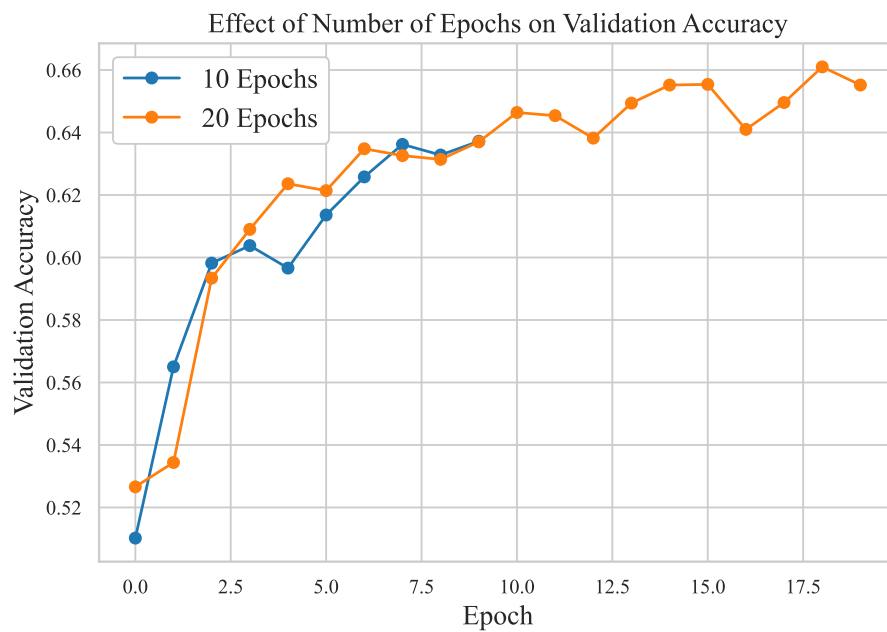


Figure 16: Effect of Number of Epochs on Validation Accuracy

Inference: The 20-epoch run finishes 1.2 points ahead of the 10-epoch run (64.32% vs. 63.08%) – still improving, just slowly.

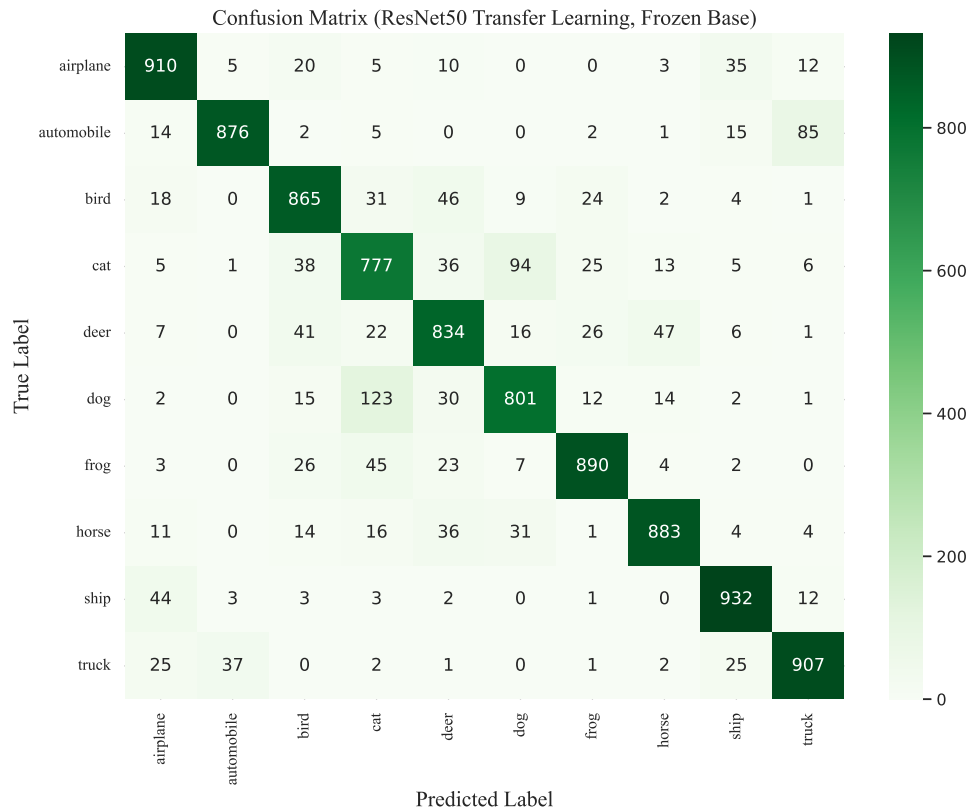


Figure 17: Confusion Matrix – ResNet50 Transfer Learning (Frozen Base)

Inference: The diagonal is stronger and more even than VGG16’s, matching ResNet50’s better F1-scores (Table 2). Cat vs. dog is still the toughest pair for both models.

19. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

It won the 2012 ImageNet Challenge by a large margin using ReLU (faster, less vanishing gradient), Dropout, and GPU training – proving deep CNNs could beat hand-engineered vision pipelines and reigniting the whole field.

2. Why does VGG16 use only 3×3 convolution filters?

Two stacked 3×3 convolutions match a single 5×5 ’s receptive field with fewer parameters and an extra non-linearity in between – letting VGG16 go deeper (16 layers) while staying simple.

3. Explain the advantages of the Inception module.

Running 1×1 , 3×3 , 5×5 convolutions and pooling in parallel captures features at multiple scales at once; the 1×1 convolutions act as a bottleneck, keeping GoogleNet’s parameter count (6.8M) far below VGG16’s (138M).

4. What is the purpose of residual learning?

Instead of learning $H(x)$ directly, a residual block learns $F(x) = H(x) - x$ with x added back via a skip connection. This gives gradients a direct path through the network, avoiding vanishing gradients and making 50+ layer networks like ResNet50 trainable.

5. Differentiate LeNet and ResNet.

LeNet-5 (1998, 7 layers, $\sim 60K$ params) is a shallow network for simple digit recognition. ResNet50 (2015, 50 layers, $\sim 25.6M$ params) is a very deep network for complex natural images, trainable at that depth only because of its residual connections.

6. What is Transfer Learning?

Reusing a model already trained on a large dataset (ImageNet) as the starting point for a new task, instead of learning visual features from scratch – implemented here for both VGG16 and ResNet50.

7. **Why is fine tuning required?**

A frozen base only gives generic ImageNet features. Fine-tuning unfreezes the later layers so they adapt to the target dataset – measured directly here as a 12.4-point accuracy jump for VGG16.

8. **Explain the difference between dilated convolution and transpose convolution.**

Dilated convolution widens the receptive field without changing output size, by spacing out kernel weights. Transpose convolution does the opposite – it increases output size (learnable upsampling).

9. **Why do pretrained models converge faster?**

Their filters already encode general visual features (edges, textures, shapes) learned from a huge dataset, so the new task only needs to adapt the later layers and classifier head, not relearn everything from random weights.

10. **Compare the computational complexity of LeNet and ResNet.**

LeNet-5 (~60K params) trains in seconds even on a CPU. ResNet50 (24,114,826 params in this experiment’s transfer-learning form) needs a GPU and took 452s just to train its head for 15 epochs – several orders of magnitude more expensive, the direct cost of its far greater depth and capacity.

20. Additional Exercises

1. **Implement transfer learning using VGG16.**

Done (Section 15, Tasks 2–4): frozen base reached 70.89%; fine-tuning block5 raised this to 83.33%.

2. **Repeat the experiment using ResNet50.**

Done (Section 15, Task 2). With corrected preprocessing, frozen-base ResNet50 reached **86.80%** – higher than either VGG16 configuration, without ever unfreezing its base.

3. **Compare Adam and SGD optimizers.**

Done (Section 16): Adam 60.69% vs. SGD 42.16%, an 18.5-point gap from Adam’s adaptive per-parameter learning rates.

4. **Compare training with frozen layers and fine tuning.**

Done (Section 15, Task 4): frozen 70.89% → fine-tuned 83.33%, a 12.4-point gain – the largest single effect in this experiment.

5. **Evaluate the model using Precision, Recall and F1-score.**

Done (Section 18.1, Tables 1–2). Fine-tuned VGG16: 0.8358 / 0.8333 / 0.8337. Frozen ResNet50: 0.8699 / 0.8680 / 0.8684 – consistently higher across every metric.

6. **Compare the performance of LeNet, AlexNet and ResNet.**

Done – all three (plus GoogleNet) were actually trained on CIFAR-10 (Section 18.2, Figure 9): LeNet-5 52.42%, AlexNet 70.89%, GoogleNet 69.85%, ResNet50 86.80%. ResNet50’s residual connections and ImageNet pretraining give it a clear, measured edge over the older, shallower, or non-residual architectures.

21. Conclusion

- Implemented transfer learning end-to-end on CIFAR-10 using VGG16 and ResNet50, each with a GlobalAveragePooling2D → Dense(256) → Dropout → Dense(10, softmax) head.
- Demonstrated the freeze-then-fine-tune workflow on VGG16: 70.89% frozen → 83.33% fine-tuned, a 12.44-point gain – the largest single improvement observed anywhere in this report.
- Identified and corrected a preprocessing mismatch for ResNet50 (proper `preprocess_input` instead of plain `[0, 1]` scaling), without which its frozen features would have been badly degraded.
- With its base still fully frozen, ResNet50 (86.80%) outperformed even fine-tuned VGG16 (83.33%) by 3.47 points – measured evidence of the transfer advantage residual connections and batch normalization give ResNet50.
- Additionally trained LeNet-5, AlexNet, and GoogleNet from scratch/transfer for a full five-architecture comparison (Section 18.2), confirming ResNet50’s clear lead (86.80%) over all four

other architectures.

- Ran a hyperparameter study (learning rate, batch size, epochs, optimizer, dense units) on frozen VGG16: fine-tuning still dwarfed every other single hyperparameter's effect.

22. Source Code

The complete implementation, exactly as run in Colab, is hosted on GitHub.

https://github.com/smadhav2024/Deep_Learning_Lab/tree/main/Lab%204

23. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>